

ESTRUCTURAS DE DATOS

Ayudantía 4: TDAs Lineales

Ignacia Nettle Oliveri
inettle@usm.cl

BLOQUE 1

Listas

Listas doblemente enlazadas

Listas Doblemente Enlazadas

Definición

Las listas doblemente enlazadas son una variante de las listas enlazadas en donde, además del puntero al nodo siguiente, almacenan un puntero **al nodo anterior** de la lista.

Esta modificación facilita la implementación de algunos métodos del TDA `tLista`.

En específico, hace que el método `prev()` se ejecute en tiempo $O(1)$, en lugar de $O(n)$ como en las listas enlazadas.

```
struct Nodo{  
    tElem info;  
    Nodo *ant;  
    Nodo *sig;  
}
```



Listas Doblemente Enlazadas

Ejemplo

```
int tLista::erase(){
    if(curr == tail) return NULL;
    tElem item = curr->sig->info;
    Nodo *temp = curr->sig;
    curr->sig->sig->ant = curr;
    curr->sig = curr->sig->sig;
    delete temp;
    listSize--;
    return item;
}
```

Desventajas

La única desventaja de las listas doblemente enlazadas es que ocupan el **doble de espacio** que una lista enlazada simple, ya que por cada nodo necesitamos dos punteros en lugar de uno.

Esto quiere decir que, para listas demasiado grandes, **especialmente** es más eficiente usar una lista enlazada simple.

Listas Doblemente Enlazadas: Tiempos de Ejecución

Método	Tiempo de Ejecución
<code>clear</code>	$O(n)$
<code>insert</code>	$O(1)$
<code>append</code>	$O(1)$
<code>erase</code>	$O(1)$
<code>moveToStart</code>	$O(1)$
<code>moveToEnd</code>	$O(1)$

Método	Tiempo de Ejecución
<code>next</code>	$O(1)$
<code>prev</code>	$O(1)$
<code>length</code>	$O(1)$
<code>currPos</code>	$O(1)$
<code>moveToPos</code>	$O(n)$
<code>getValue</code>	$O(1)$

BLOQUE 2

Ejercicios

Ejercicio 1

Sobre el TDA enlazada simple, se han hecho algunas modificaciones: la lista se encuentra ordenada en forma **ascendente** y sobre el struct `Nodo` se realizó la siguiente modificación:

```
struct Nodo{  
    tElem info;  
    Nodo *sig;  
    Nodo *skip;  
}
```

`Nodo *skip` se utiliza para saltar una cantidad `k` de nodos desde el nodo actual.

Dado el TDA de lista enlazada `tLista` utilizando esta nueva definición de `Nodo`, implemente el método `buscar(tElem x)`, que busca el elemento `x` en la lista y retorna `true` si `x` está en la lista y `false` en otro caso.

El tiempo de ejecución del algoritmo debe ser $O(\frac{n}{k} + k)$.

Ejercicio 1 - Solución

```
bool buscar(tElem x){
    Nodo auxCurr = head->sig;
    Nodo auxPrev = head;
    while(auxCurr->info <= x && auxCurr != tail){
        auxPrev = auxCurr;
        auxCurr = auxCurr->skip;
    }

    while(auxPrev != auxCurr){
        if(auxPrev->info == x) return true;
        auxPrev = auxPrev->sig;
    }
    return false;
}
```


Ejercicio 2

Dos hechizos de invocación han chocado, produciendo dos listas doblemente enlazadas de energía **separadas**. Para estabilizar el núcleo, se deben fusionar insertando la segunda lista dentro de la primera de manera **intercalada**, pero leyendo la segunda lista de **atrás hacia adelante**.

Implementa la función `fusionarReversa` que reciba dos TDAs `tLista` (L1 y L2). Por cada elemento que avance L1, se debe insertar un elemento extraído desde el final de L2. La función debe retornar la lista fusionada.

Por ejemplo: L1 = [A, B, C], L2 = [X, Y, Z]. La salida debe ser: [A, Z, B, Y, C, X].

Ejercicio 2 - Solución

```
tLista fusionarReversa(tLista L1, tLista L2){
    L1.moveToStart();
    L2.moveToEnd();

    if(L2.length() > 0) L2.prev();

    for(int i = 0; i < L2.length(); i++){
        L1.next(); // avanzamos para intercalar
        L1.insert(L2.getValue()); // insertamos elemento de L2
        L1.next(); // saltamos elemento recién insertado
        L2.prev(); // retrocedemos en L2
    }
    return L1;
}
```

Ejercicio 3

Se tiene un mapa donde se han identificado un conjunto de elementos. Cada elemento está representado en el mapa por un punto (x, y) . Suponga que representaremos los elementos del plano usando el TDA `tLista`. Implemente la función:

```
tLista* estanAfuera(tLista *l, tPunto supIzq, tPunto infDer)
```

que recibe como parámetros una lista `l` que almacena un conjunto de puntos, y dos puntos `supIzq` e `infDer`, los cuales representan las coordenadas superior izquierda e inferior derecha de un rectángulo en el plano. Los lados del rectángulo son paralelos a los ejes de coordenadas.

La función debe retornar una lista con todos los puntos del conjunto que **NO** están dentro de dicho rectángulo.

Suponga que existe un TDA `tPunto`, el cual tiene los métodos `getX()` y `getY()`, las cuales devuelven las coordenadas x e y respectivamente de un punto como `float`.

Ejercicio 3 - Solución

```
tLista* estanAfuera(tLista *l, tPunto supIzq, tPunto infDer){
    tLista *r = new tLista;
    for(l->moveToStart(); l->currPos() < l->length(); l->next()){
        tPunto p = l->getValue();
        if(supIzq.getX() > p.getX() || p.getX() > infDer.getX()){
            r->append(p);
        } else if(infDer.getY() > p.getY() || p.getY() > supIzq.getY()){
            r->append(p);
        }
    }
    return r;
}
```

CIERRE

¡Eso es todo 🧐!